Week 5 - Ethical Hacking & Exploiting Vulnerabilities

1.0 Overview

The objective of Week 5 was to learn and apply ethical hacking techniques, exploit vulnerabilities in the Juice Shop test environment, and implement security fixes for the discovered vulnerabilities. The three main areas of focus were Reconnaissance on the target web application, SQL Injection discovery and remediation and Cross-Site Request Forgery (CSRF) protection

2.0 Ethical Hacking Basics & Reconnaissance

2.1 What is Reconnaissance?

Reconnaissance is the first phase of ethical hacking. It involves gathering as much information as possible about the target system before attempting any exploitation. This information helps identify potential attack surfaces, exposed services, and misconfigurations.

2.2 Nmap Service Scan

2.2.1 Nmap Command and Analysis

Nmap (Network Mapper) is a widely used open-source tool for network discovery and security auditing. It can identify open ports, running services, and software versions.

The following command was used to perform a service scan on the Juice Shop application:

nmap -sV -sC -p 3000 localhost -oN recon_nmap.txt

(Figure 2.2.1 - nmap output showing port 3000 open with security headers visible*)*

The Nmap scan was performed using several options to gather detailed information about the target service running on port 3000. The -sV flag was used to detect the version of the service running on the port, helping identify the specific software and its release. The -sC flag enabled Nmap's default scripting engine scripts, which perform basic enumeration and security checks. The -p 3000 option limited the scan to port 3000, focusing only on the service of interest rather than scanning all ports. Finally, the -oN flag was used to save the scan results in a normal, human-readable format to an output file for later analysis and documentation.

2.2.2 What the Scan Verified

The Nmap scan results showed that port 3000 was open and running an HTTP service associated with Express.js. Although Nmap identified the service as *ppp?*, further inspection of the HTTP response confirmed that the application was an Express.js web server.

Content-Security-Policy header was active, helping to mitigate content injection attacks. The *Strict-Transport-Security* header was present, enforcing secure HTTPS connections, while the *X-Frame-Options: DENY* header prevented the application from being embedded in frames, reducing the risk of clickjacking attacks. Additionally, the *Access-Control-Allow-Methods* header indicated that CORS had been configured to permit the HTTP methods GET, POST, PUT, DELETE, and PATCH.

These findings confirmed that the required security headers were successfully applied and functioning as intended.

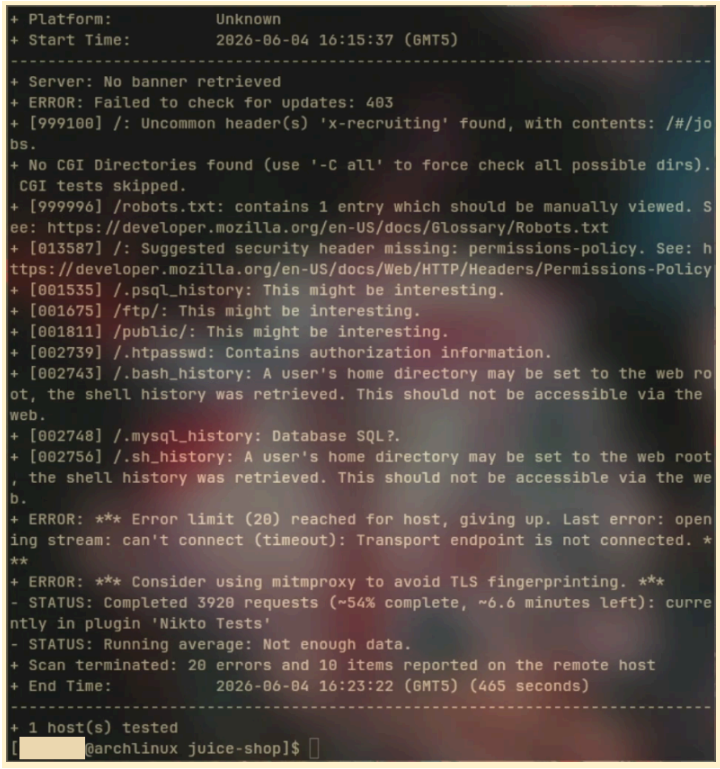
2.3 Nikto Web Scan

Nikto is an open-source web server scanner that performs comprehensive tests against web servers for dangerous files, outdated software, and other security issues.

The following command was used:

nikto -h http://localhost:3000 2>&1 | tee recon_nikto.txt

The assessment identified several security findings of varying severity. A medium-risk issue was the exposure of the */ftp/* directory, where directory listing was enabled, allowing users to view the contents of the directory. Additional medium-risk findings included public access to the */.htpasswd* file, which may contain authorization-related information, as well as the */.bash_history* and */.mysql_history* files, which could expose shell command history and database query history respectively. A low-risk finding was the absence of the *Permissions-Policy* header, indicating that browser feature permissions were not fully configured. Finally, the *robots.txt* file disclosed the existence of the */ftp* directory, which, while informational in nature, could assist attackers in identifying potentially sensitive locations within the application. These findings highlight several areas where information disclosure risks could be reduced through improved access controls and security configuration.



(Figure 2.3.1 - Nikto output showing findings and early termination due to rate limiting)

Note: The Nikto scan was rate-limited and terminated early due to the express-rate-limit middleware implemented in Week 4. This confirms that the rate limiting is functioning correctly - it was generating 429 responses that caused Nikto's connection to time out.

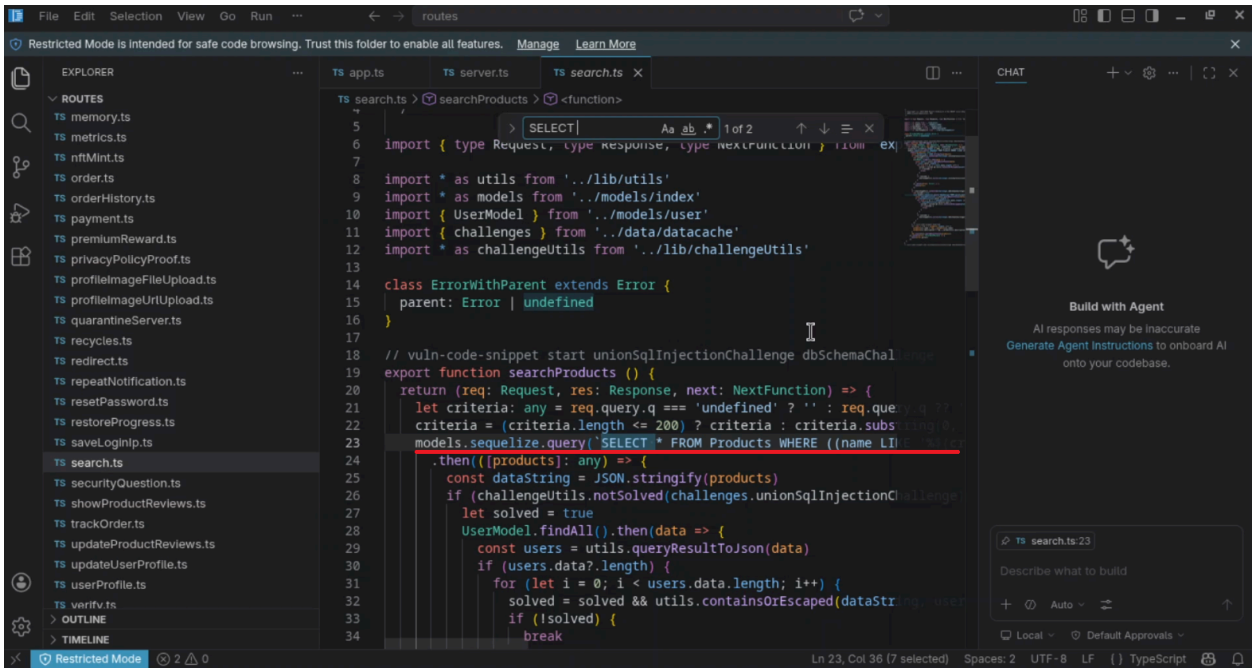
3.0 SQL Injection

3.1 What is SQL Injection?

SQL Injection (SQLi) is a web security vulnerability where an attacker inserts or "injects" malicious SQL code into a query. If the application fails to properly validate or sanitise user input, the injected SQL can manipulate the database - allowing attackers to bypass authentication, retrieve sensitive data, modify records, or in some cases execute commands on the database server.

3.2 Identifying the Vulnerable Endpoint

The product search endpoint was identified as a potential target for SQL injection. Examining the source code in routes/search.ts revealed the following vulnerable query:



(Figure 3.2.1)

models.sequelize.query(
`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%'
OR description LIKE '%${criteria}%') AND deletedAt IS NULL)
ORDER BY name`

)

The criteria variable - taken directly from the user's query string (req.query.q) - was inserted into the SQL query using string interpolation without any sanitisation or parameterization. This is a classic SQL injection vulnerability.

3.3 Exploitation

To confirm the vulnerability, the following payload was used:

curl -g "http://localhost:3000/rest/products/search?q='))--"

This payload works by using *'))* to close the open *LIKE* clause and terminate the *WHERE* condition in the SQL query. The -- sequence then comments out the remainder of the query, including the *AND deletedAt IS NULL* filter, allowing the tester to observe whether the application is vulnerable to SQL injection.

Result: The injection broke out of the *LIKE* clause and returned *ALL* products from the database, including soft-deleted items (products with deletedAt set) that should be hidden from users. This confirmed successful SQL injection.

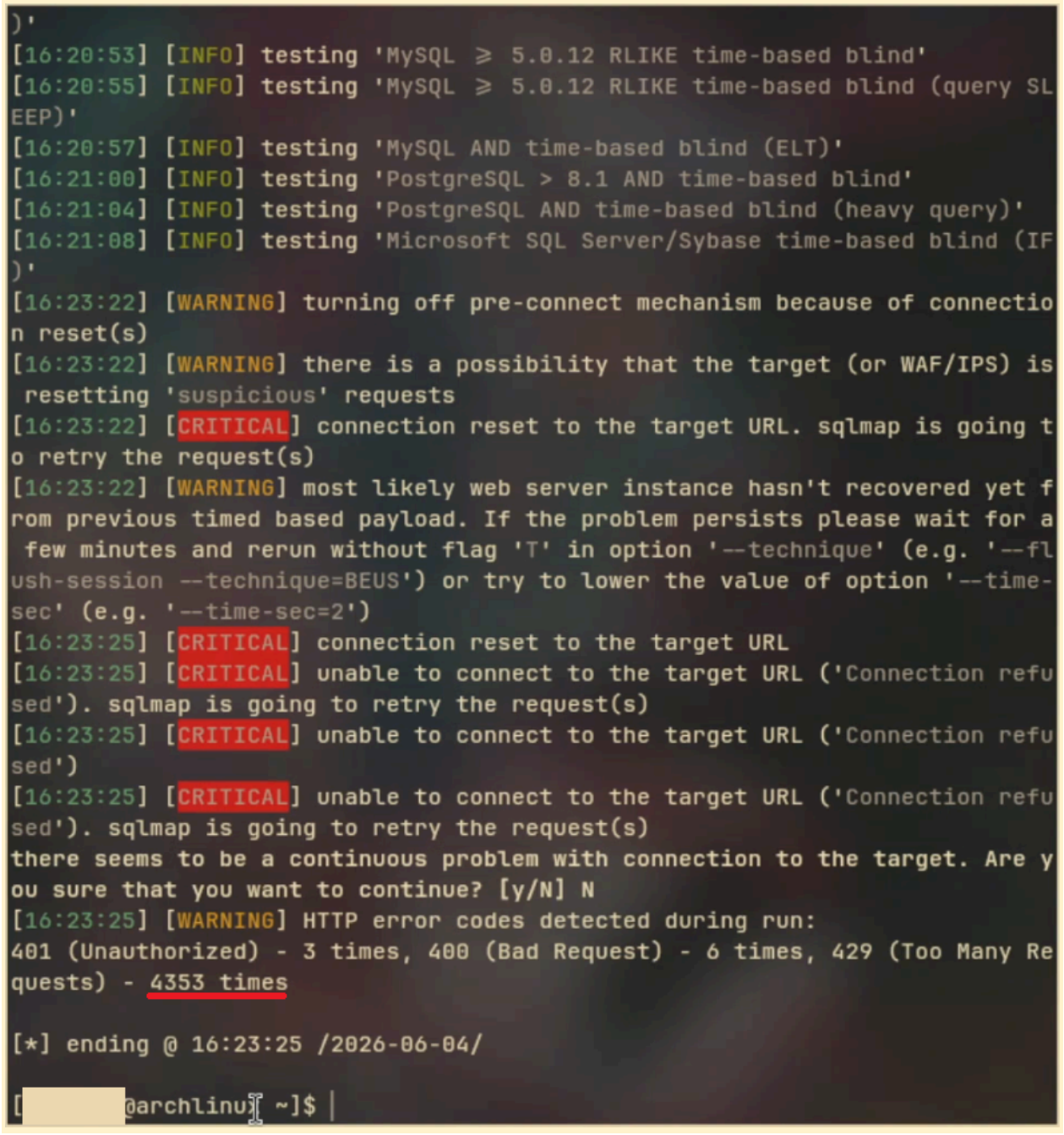
[---------@archlinux ~]$ curl -g "http://localhost:3000/rest/products/search?q='))--"
{"status":"success","data":[{"id":1,"name":"Apple Juice (1000ml)","description":"The all-time classic.","price":1.99…

And the list goes on - the string of data printed is large.

3.4 SQLMap Scan

SQLMap is an open-source penetration testing tool that automates the detection and exploitation of SQL injection vulnerabilities.

```
sqlmap -u "http://localhost:3000/rest/products/search?q=test" \
  --level=3 --risk=2 --batch --ignore-code=401 \
  --output-dir=./sqlmap_output
```

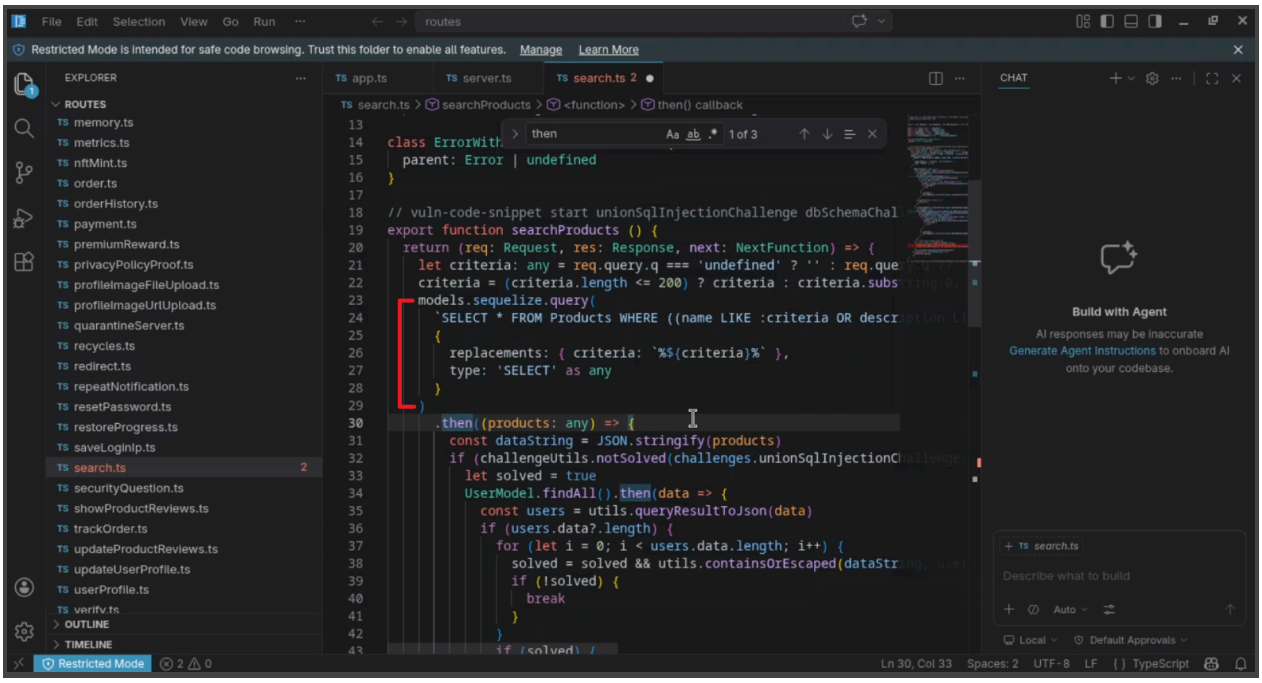


(Figure 3.4.1 - SQLMap output showing 429 responses blocking automated testing on login endpoint)

Note: When SQLMap was run against the login endpoint /rest/user/login, it was blocked by the Week 4 rate limiter with 429 responses (4353 times), confirming that the brute-force protection was working effectively. The search endpoint was confirmed injectable via manual testing above.

3.5 Fix Applied

The vulnerable string interpolation was replaced with a parameterized named query in routes/search.ts:



(Figure 3.3.1 - Fixed Code)

The line was also fixed, using the code in the highlighted text, after both the SQLMap and Nikto scans were run. I also removed the [] brackets on products in the line right after the highlighted code.

```
models.sequelize.query(
  `SELECT * FROM Products WHERE ((name LIKE :criteria
  OR description LIKE :criteria) AND deletedAt IS NULL)
  ORDER BY name`,
  {
    replacements: { criteria: `%${criteria}%` },
    type: 'SELECT' as any
  }
)
```

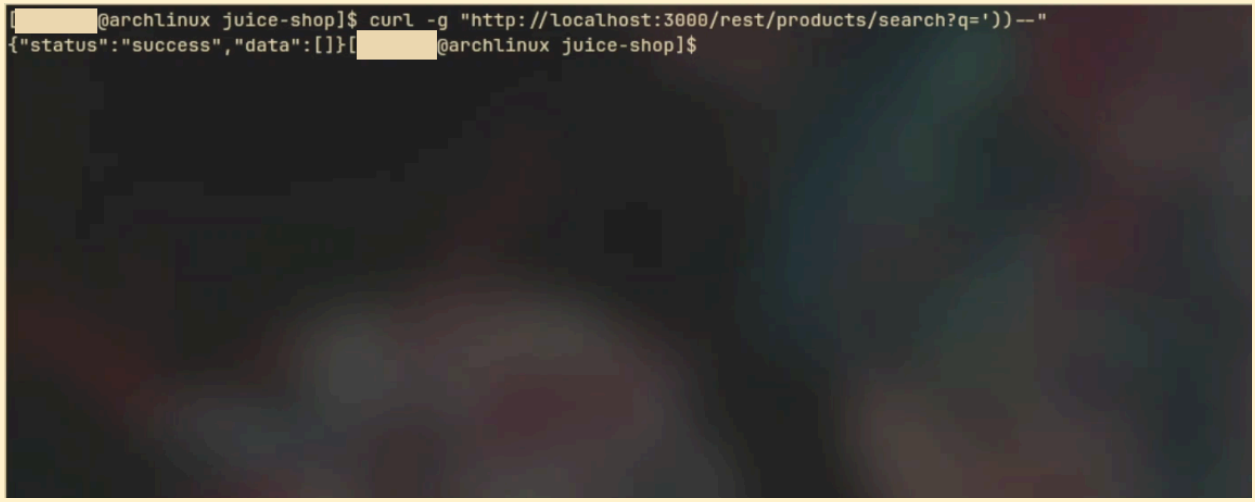
By using named replacements (:criteria), the user input is treated as data rather than executable SQL code. The database driver handles escaping automatically, preventing injection.

3.6 Verification

After rebuilding and restarting the server, the same payload was tested again:

curl -g "http://localhost:3000/rest/products/search?q='))--"

Result: {"status":"success","data":[]} - the injection payload returned an empty array instead of all products, confirming the fix was successful.



(Figure 3.3.2 - curl output showing empty result for injection payload after fix)

Normal search was also verified to still work:

curl -g "http://localhost:3000/rest/products/search?q=apple"

(Figure 3.6.1 - curl output showing normal search results still working)

Result: Only apple-related products were returned.

4.0 Cross-Site Request Forgery (CSRF) Protection

4.1 What is CSRF?

Cross-Site Request Forgery (CSRF) is an attack that tricks a victim's browser into making an unwanted request to a web application in which they are authenticated. Because the browser automatically includes cookies with every request, a malicious website can cause a logged-in user to unknowingly perform actions - such as changing their password or transferring funds - without their knowledge.

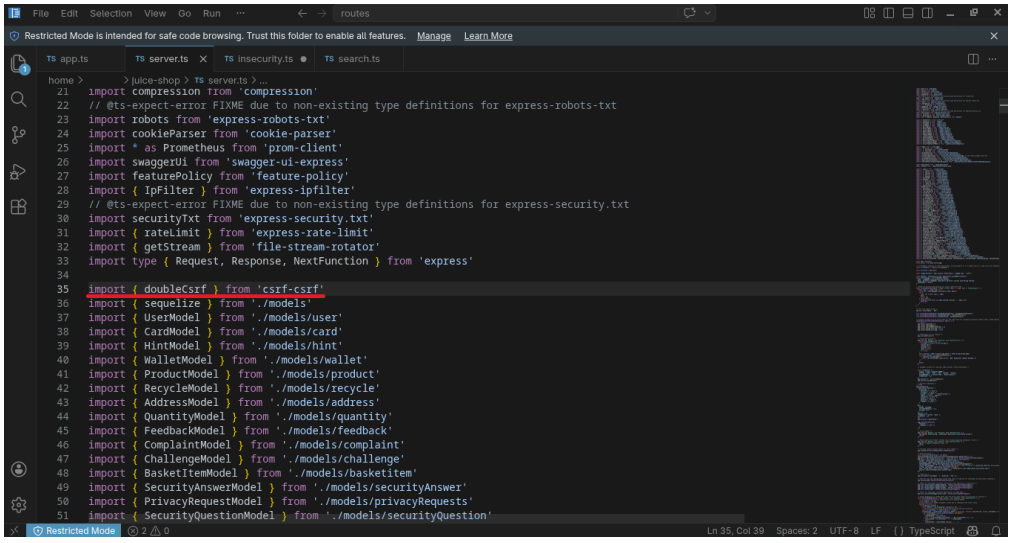
4.2 Implementation

The csurf package has been officially deprecated. The modern replacement csrf-csrf was used instead, which implements the double-submit cookie pattern - a widely accepted CSRF prevention technique.

Installation:

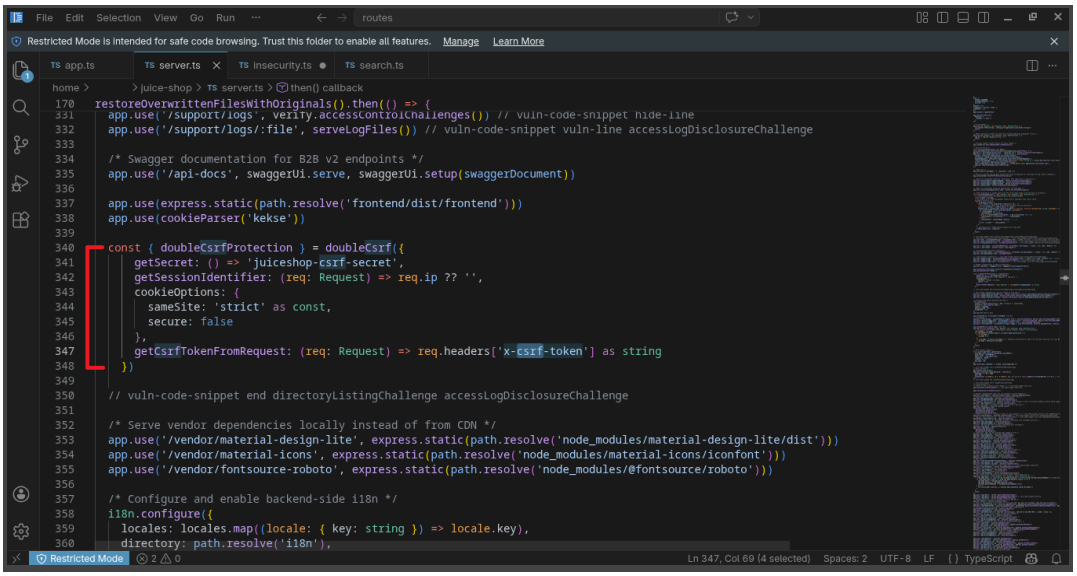
npm install csrf-csrf

Configuration added to server.ts:



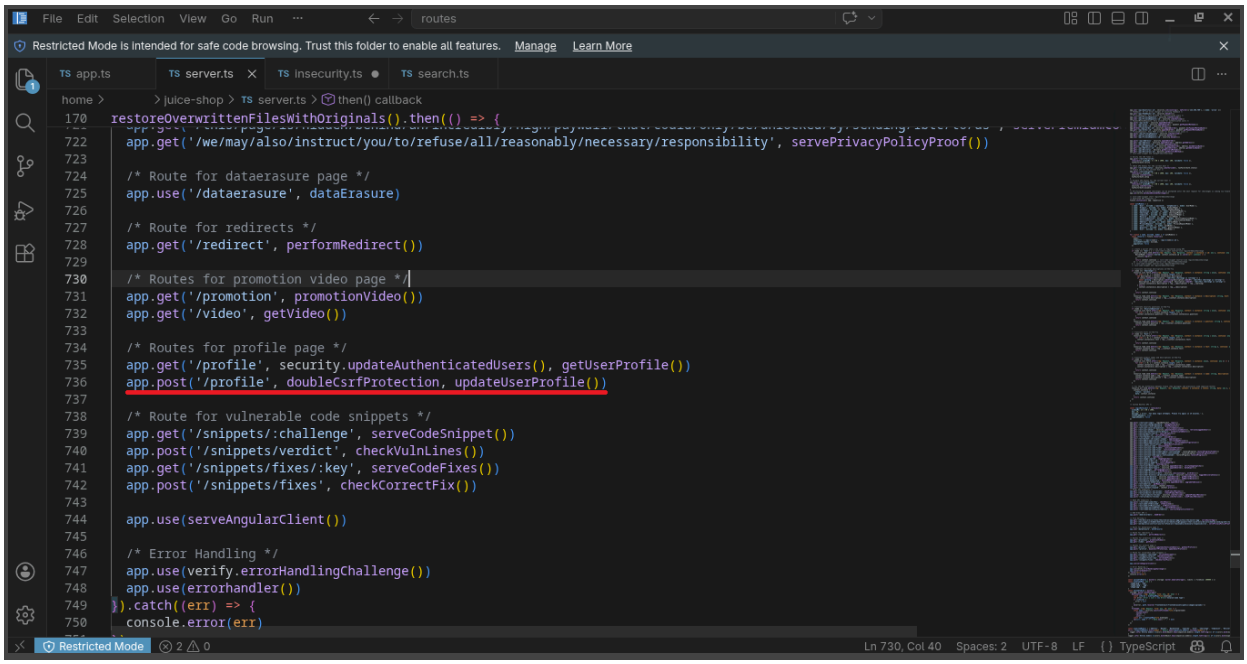
(Figure 4.2.1)

import { doubleCsrf } from 'csrf-csrf'



(Figure 4.2.2)

```
const { doubleCsrfProtection } = doubleCsrf({
  getSecret: () => 'juiceshop-csrf-secret',
  getSessionIdentifier: (req: Request) => req.ip ?? '',
  cookieOptions: {
    sameSite: 'strict' as const,
    secure: false
  },
  getCsrfTokenFromRequest: (req: Request) => req.headers['x-csrf-token'] as string
})
```



(Figure 4.2.2)

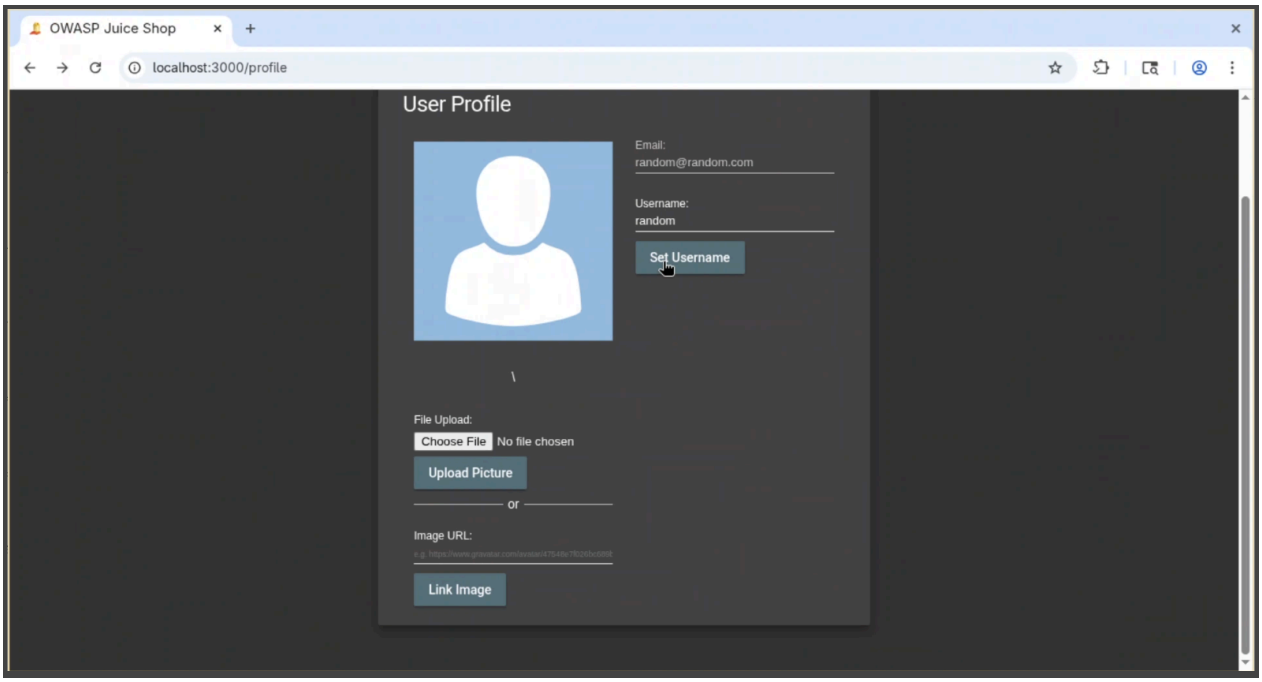
app.post('/profile', doubleCsrfProtection, updateUserProfile())

The double-submit cookie pattern protects against CSRF attacks by requiring the same token to be sent in both a cookie and a request header. When a user loads the application, the server issues a CSRF token as a cookie. For subsequent requests, the client must include this token in the x-csrf-token header. The server then verifies that the token in the header matches the token stored in the cookie. Since a cross-site attacker cannot read the victim's cookie due to the browser's same-origin policy, they cannot generate a matching header token, causing malicious requests to be rejected.

4.3 Testing with Burp Suite

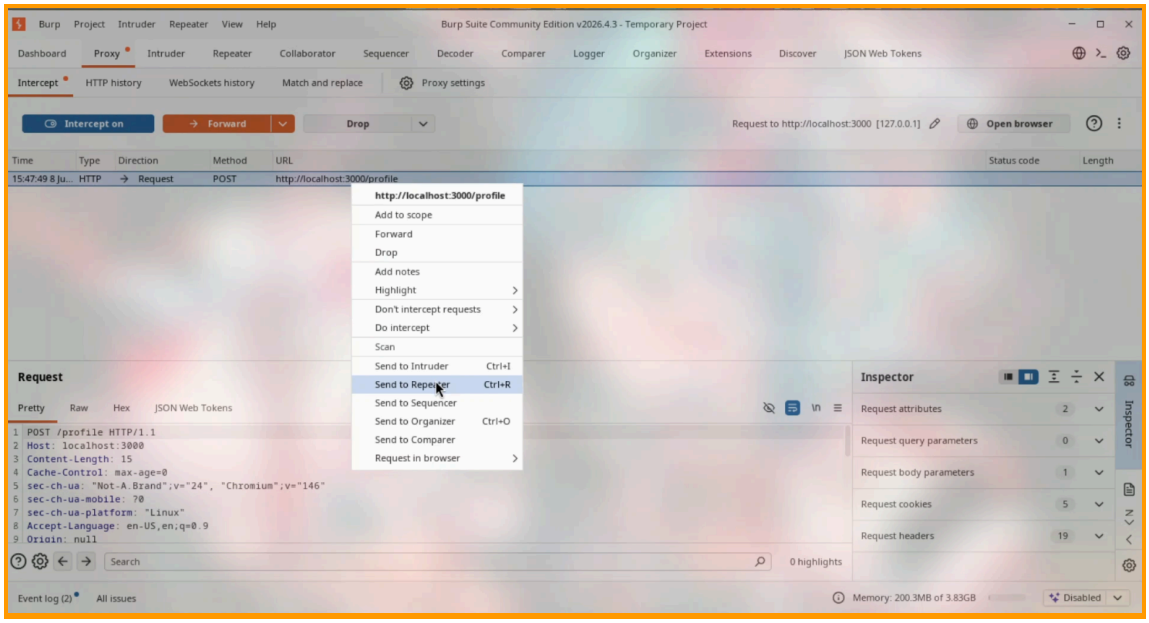
Burp Suite Community Edition was used to intercept and modify a legitimate POST /profile request to test CSRF protection.

To test the CSRF protection mechanism, the browser was configured to use a proxy at *127.0.0.1:8080*. After logging into Juice Shop, the profile page was accessed and the username was changed:



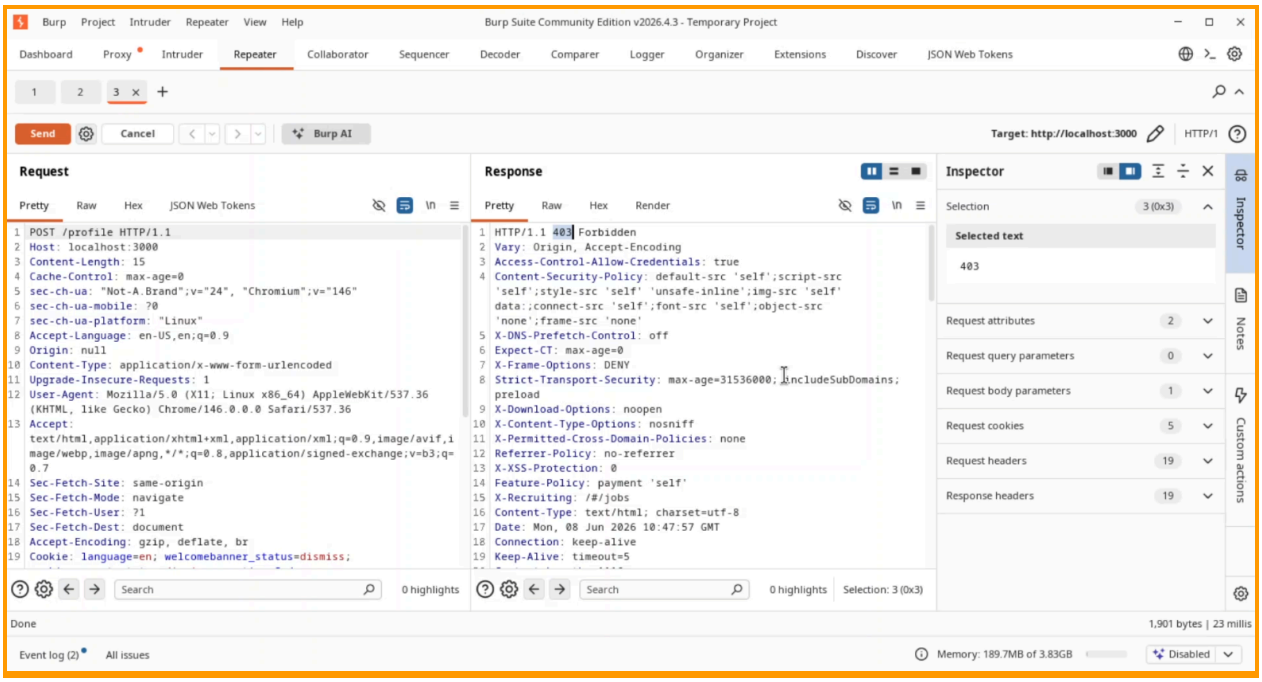
(Figure 4.3.1 - Setting my username to "random")

causing a *POST /profile* request to be generated:



(Figure 4.3.2 - POST /profile request about to be sent to Burp Suite Repeater)

This request was captured in Burp Suite through the *Proxy* to *HTTP History* tab and then sent to *Repeater* for modification. The *x-csrf-token header* was removed from the request before it was resent to the server, allowing verification of whether the application properly enforced CSRF token validation.



(Figure 4.3.3 - Burp Suite response showing 403 Forbidden without CSRF token)

Result without CSRF token:

HTTP/1.1 403 Forbidden

The server rejected the request with 403 Forbidden, confirming that CSRF protection was functioning correctly.

4.4 Note on JWT API Endpoints

The JSON API endpoints *(/api/Users, /rest/user/login*, etc.) use JWT Bearer token authentication. These endpoints are inherently resistant to CSRF because browsers do not automatically include Authorization headers in cross-origin requests. CSRF protection is therefore most critical for cookie and session-based form routes such as *POST /profile*.